

Day 10: LLM Inference & Alignment

- **LLM Inference & Alignment** — From a base model to a usable assistant
- 3 hours lecture + practical
- Slides, notes, and code on the course site

- Autoregressive Decoding
- KV Cache & Memory
- Sampling & Systems
- From GPT to ChatGPT (light)

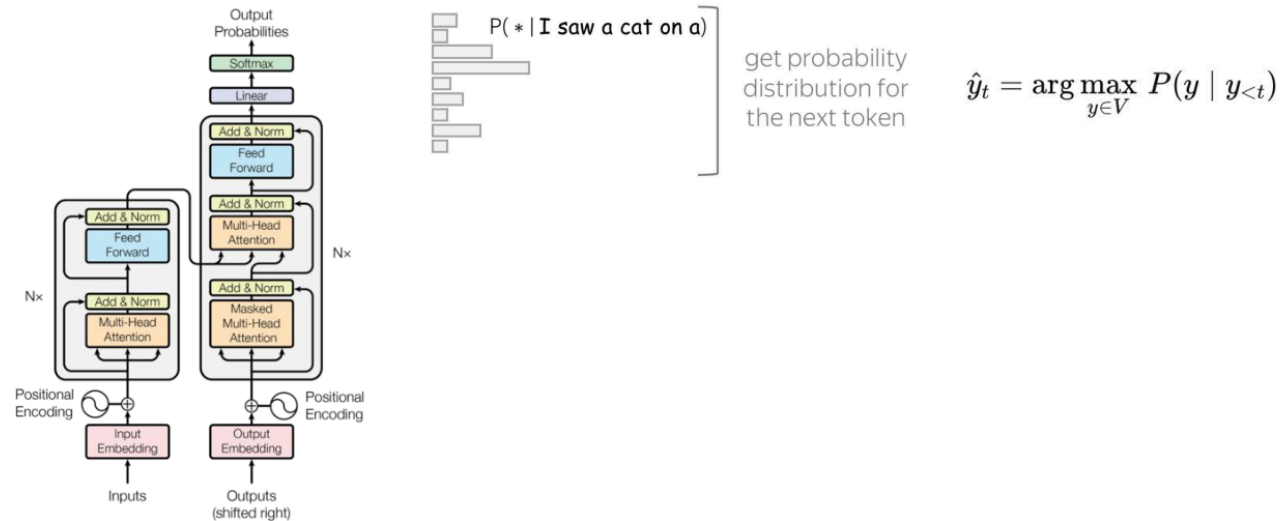
1 · Autoregressive Decoding

1.1 The Decode Loop

- Day 9 gave us $p_{\theta}(x_t \mid x_{<t})$ — now we **generate**
- Sample x_t , append it, feed back, repeat
- Inherently **serial**: token t needs tokens $< t$
- Two phases: prefill (the prompt) then decode (one by one)
- Latency = time-to-first-token + per-token time

1.1 The Decode Loop

At generation time - One token at a time



https://lena-voita.github.io/nlp_course/language_modeling.html

1.2 Exposure Bias

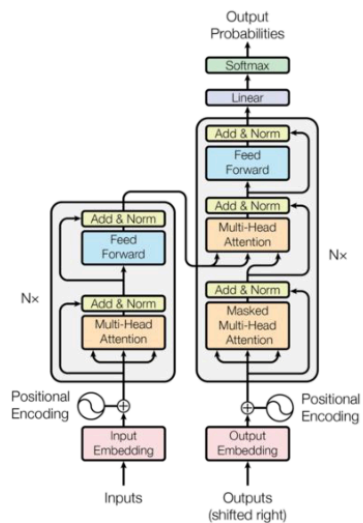
- Training: teacher forcing on ground-truth context
- Generation: model conditions on its **own** past outputs
- Small errors can compound over a long generation
- Mitigated by scale, good data, and decoding choices
- A reason sampling strategy matters at inference

1.3 Greedy, Sampling, and Beam

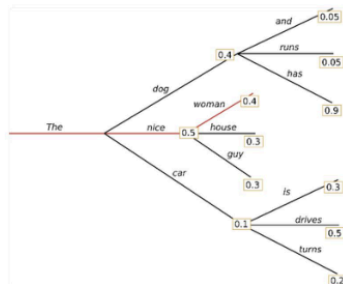
- Greedy: take the argmax each step — repetitive, brittle
- Sampling: draw from the distribution — diverse
- Beam search: keep top- k partial sequences (translation legacy)
- Beam helps short, high-precision outputs; hurts open-ended text
- Modern chat: temperature + nucleus sampling

1.3 Greedy, Sampling, and Beam

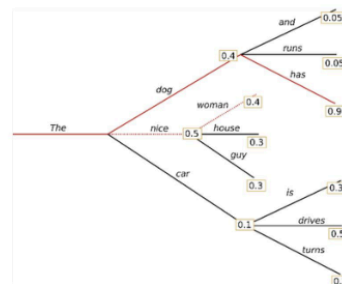
At generation time - Search [Beam]



Greedy



Beam



2 · KV Cache & Memory

2.1 Why Decode Is Quadratic Without a Cache

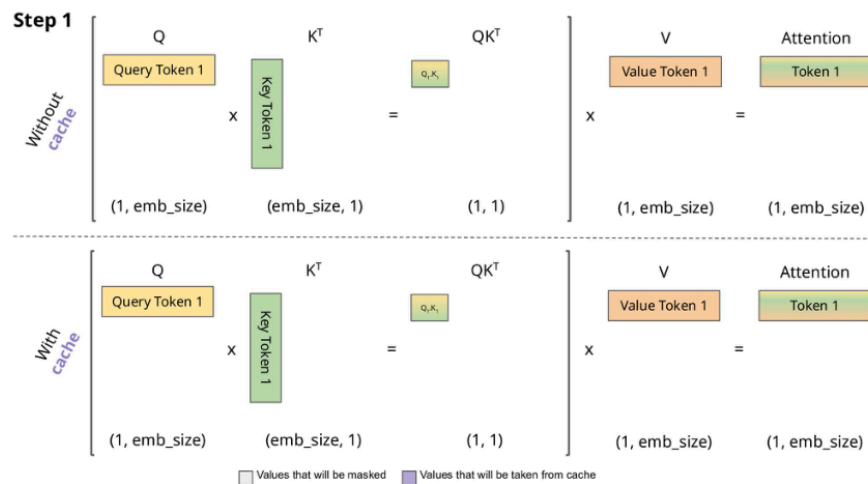
- Naive decode recomputes attention over the whole prefix
- Step t costs $O(t)$; a full sequence costs $O(T^2)$
- But keys/values of past tokens never change
- Idea: compute them once, store, and reuse
- This is the single biggest inference optimization

2.2 The KV Cache

- Cache K, V for every layer and past position
- Each new token computes its own q, k, v
- Append k, v to the cache; attend over the cache
- Per-step cost drops from $O(t)$ recompute to $O(1)$ append
- Prefill fills the cache; decode extends it

2.2 The KV Cache

KV cache



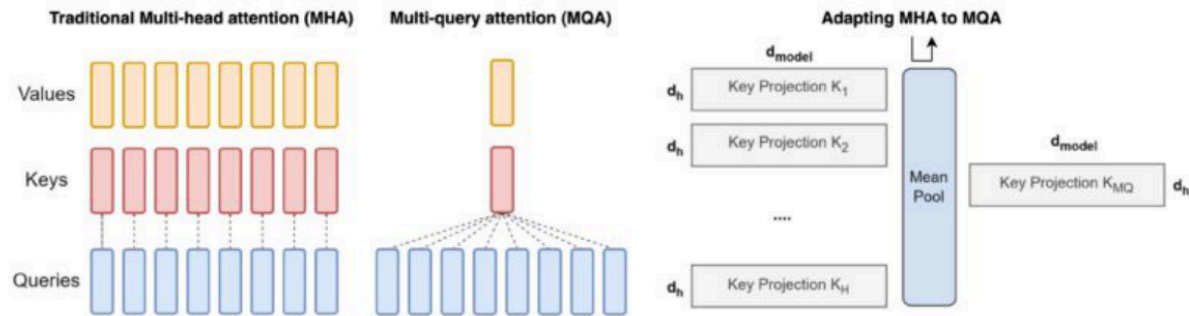
<https://huggingface.co/blog/not-lain/kv-caching>

2.3 KV-Cache Memory and GQA

- Cache size $2 \cdot L \cdot n_{\text{kv}} \cdot d_{\text{head}} \cdot T$ per sequence
- Grows with context length T and batch size
- Often the real bottleneck — bandwidth and memory bound
- Grouped-query attention (Day 9) shrinks n_{kv}
- PagedAttention manages the cache like virtual memory

2.3 KV-Cache Memory and GQA

The Inference Bottleneck: The KV Cache Size Introduce: Multi-Query Attention



In multi-query attention, the heads for keys and values are averaged so that all query heads share the same key and value head.

Fast Transformer Decoding: One Write-Head is All You Need [Shazeer et al., 2019]

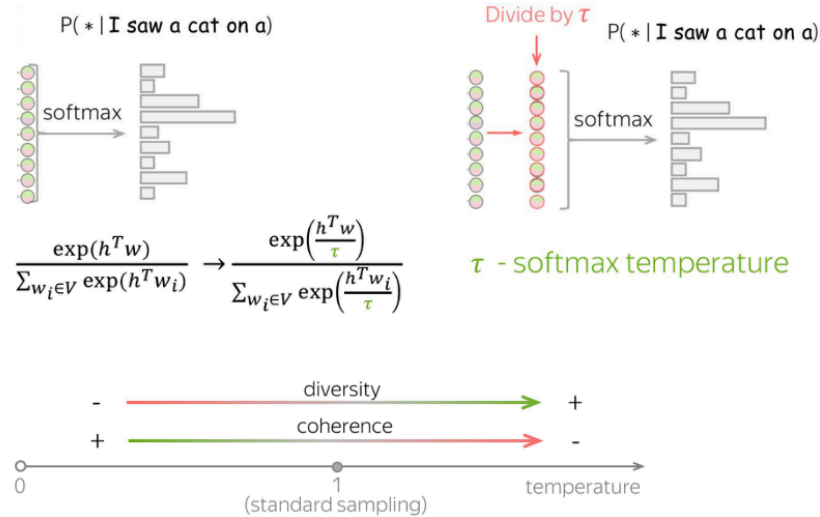
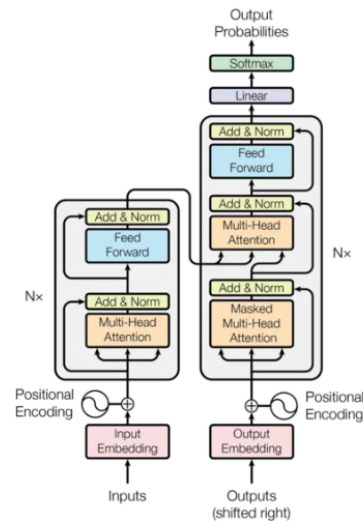
3 · Sampling & Systems

3.1 Temperature

- Rescale logits before softmax: $p_i \propto \exp\left(z_i \frac{1}{T}\right)$
- $T < 1$: sharper, more deterministic
- $T > 1$: flatter, more random
- $T \rightarrow 0$ recovers greedy decoding
- The first knob for the quality/diversity trade-off

3.1 Temperature

At generation time - One token at a time [Sampling]



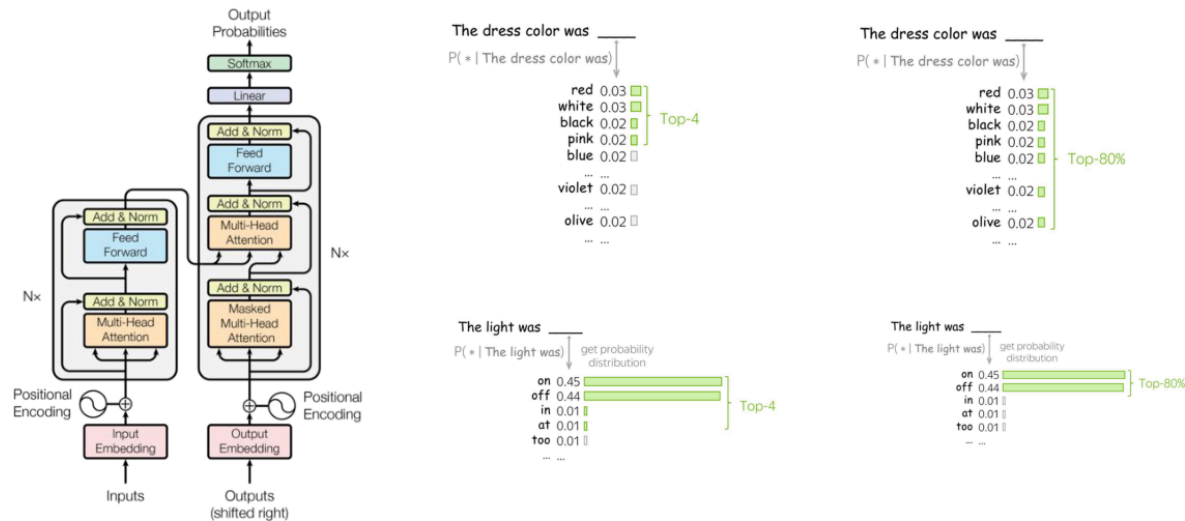
https://lena-voita.github.io/nlp_course/language_modeling.html

3.2 Top-k and Top-p (Nucleus)

- Top- k : sample only from the k most likely tokens
- Top- p : smallest set whose probability mass exceeds p
- Truncates the unreliable long tail of the distribution
- Nucleus (top- p) adapts the cutoff to each step
- Common default: temperature + top- p together

3.2 Top-k and Top-p (Nucleus)

At generation time - One token at a time [Topk, Top-p]



https://lena-voita.github.io/nlp_course/language_modeling.html

3.3 Throughput: Prefill vs Decode

- Prefill is compute-bound; decode is memory-bandwidth-bound
- Decode reuses weights for one token at a time
- Batch many requests to amortize weight reads
- Continuous batching: add/remove requests mid-flight
- Metrics: tokens/sec, TTFT, inter-token latency

3.4 FlashAttention & PagedAttention

- FlashAttention: IO-aware exact attention, no big $N \times N$ matrix
- Tiles the computation in fast on-chip memory
- Speeds up training and prefill, saves memory
- PagedAttention (vLLM): non-contiguous KV cache in pages
- Less fragmentation $\rightarrow$ higher batch sizes, more throughput

3.5 Speculative Decoding & Quantization

- Speculative: a small draft model proposes several tokens
- The big model **verifies** them in one parallel pass
- Accepted tokens are free — same distribution, fewer big-model steps
- Quantization: INT8 / INT4 weights shrink memory + bandwidth
- Both attack the serial / memory-bound decode cost

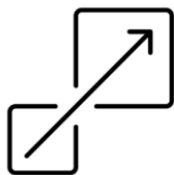
4 · From GPT to ChatGPT (light)

4.1 From Base Model to Assistant

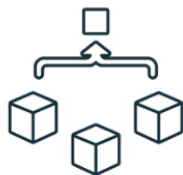
- Base LM (Day 9): predicts internet text, not ‘helpful’
- Pipeline: pretrain $\rightarrow$ SFT $\rightarrow$ preference tuning
- Then: system prompt + safety + tools
- Each stage uses far less data than pretraining
- Turns a next-token predictor into a chat model

4.1 From Base Model to Assistant

Change of Paradigm
What did transformers enable?



Scalability



Generalizability



Reasoning



Long range



Multimodal

4.2 Supervised Fine-Tuning (SFT)

- Fine-tune on curated (instruction, response) conversations
- Same next-token loss, masked to the **assistant** tokens
- Teaches format, instruction-following, tone
- LoRA / PEFT: adapt cheaply without full fine-tuning
- Small, high-quality data matters more than volume

4.3 Preference Tuning (RLHF / DPO)

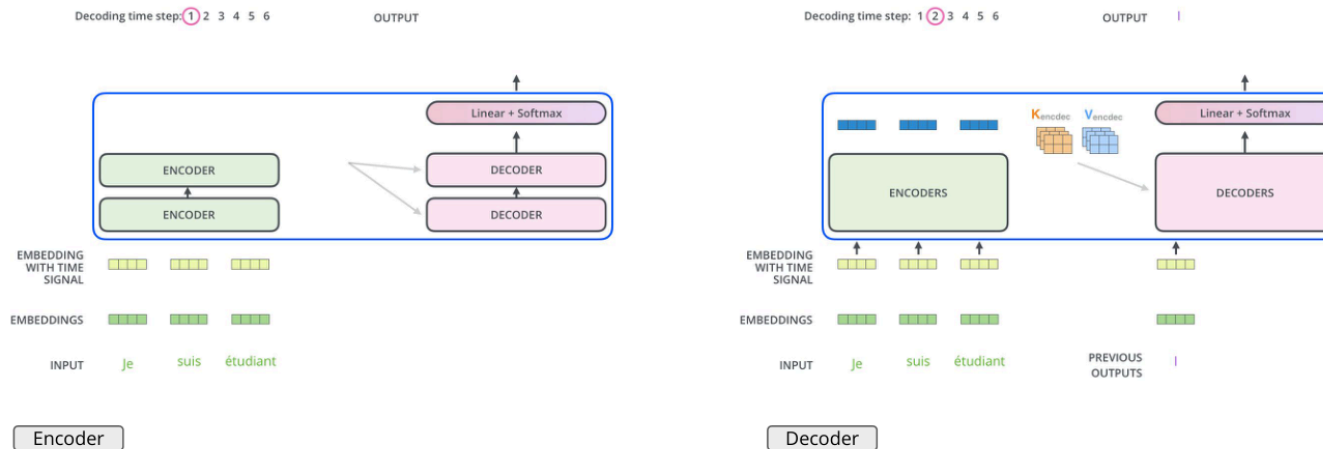
- Collect human preferences: response A vs response B
- RLHF: train a reward model, optimize it with PPO
- DPO: optimize the preferences directly, no RL loop
- Aligns the model with human judgments of quality
- Details in the optional notes — concept is the takeaway

4.4 Course Recap

- Week 1: ML/DL foundations → CNNs → Transformers
- Week 2: generative models, diffusion, and autoregressive LLMs
- Three generative paths: AR, diffusion/score, flows
- Same backbone — Transformers — powers all of it
- Transformers everywhere: text, vision, audio, biology

4.4 Course Recap

Summary Visualization



Visualization: <https://jalamar.github.io/illustrated-transformer/>

- Day 10: **LLM Inference & Alignment**
- From a base model to a usable assistant
- Questions welcome — practical follows

Questions?

Practical session follows — see course site.